BEATRIZ, BRENDA E MARIA EDUARDA B.

PROJETO INTEGRADOR - 3° TERMO

Manutenção Industrial 4.0

LIMEIRA – SP

2026

Sumário

1. PROPOSTA 2: MANUTENÇÃO INDUSTRIAL 2

2. LEVANTAMENTO DE REQUISITOS 3

2.1. REQUISITOS FUNCIONAIS 3

2.2. REQUISITOS NÃO FUNCIONAIS 3

3. PRIORIZAÇÃO 4

4. FIGMA 6

5. METODOLOGIAS ÁGEIS 7

6. README e Versionamento (GIT) 8

7. DIAGRAMA DE CLASSE 9

8. DIAGRAMA DE SEQUÊNCIA 10

9. DIAGRAMA BANCO DE DADOS 11

1. PROPOSTA 2: MANUTENÇÃO INDUSTRIAL

Situação Problema: "Na era da Indústria 4.0, a disponibilidade das máquinas é crucial para a produtividade. Paradas não planejadas custam milhares de reais e atrasam toda a cadeia produtiva. Fábricas modernas buscam migrar de anotações em papel para sistemas digitais que prevejam e registrem manutenções de forma eficiente." "O Senai contratou sua consultoria para acabar com as 'fichas de papel' penduradas nas máquinas. Eles precisam de um sistema centralizado onde a equipe de manutenção possa registrar intervenções e os gerentes possam acompanhar a saúde dos equipamentos em tempo real."

Desafio: "Sua equipe deve implementar o Back-End do 'MaintSys', uma API para controle de manutenção de maquinário industrial. O sistema deve garantir a integridade dos dados e permitir que técnicos registrem atividades através de futuros tablets ou terminais industriais."

Funcionalidades Essenciais:

1. Gestão de Técnicos e Acesso: Cadastro de técnicos especializados com autenticação segura.

2. Inventário de Máquinas: Cadastro de equipamentos (Número de Série, Modelo, Localização no Galpão, Data de Instalação).

3. Ordens de Serviço (O.S.): Criação de ordens de manutenção (Preventiva ou Corretiva), vinculando um técnico a uma máquina específica.

4. Histórico de Manutenções: Log completo de todas as intervenções realizadas em uma máquina, permitindo análise de reincidência de defeitos.

5. Alertas de Status: Sistema para notificar quando uma máquina muda de status (ex: "Máquina 04 em Parada Crítica" ou "Manutenção Concluída").

Requisitos e Restrições:

- Back-End desenvolvido em Laravel (PHP).
- Banco de dados relacional MySQL.
- Utilizar o Eloquent ORM para modelar as tabelas e relações (ex: Uma Máquina tem muitas Ordens de Serviço).
- API RESTful retornando JSON.
- Código seguindo padrões PSR e Clean Code.
- Publicação no GitHub e testes via Insomnia/Postman.

2. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos é o processo de identificar, analisar e documentar as necessidades reais dos clientes e usuários.

2.1. REQUISITOS FUNCIONAIS

- **RF1** O sistema deve coletar dados de sensores (vibração, temperatura e pressão) em tempo real.
- **RF2** O sistema deve armazenar os dados coletados em banco de dados histórico.
- **RF3** O sistema deve analisar automaticamente os dados utilizando algoritmos preditivos.
- **RF4** O sistema deve gerar alertas automáticos quando detectar anomalias.
- **RF5** O sistema deve permitir o cadastro de máquinas e equipamentos.
- **RF6** O sistema deve permitir o cadastro de usuários com diferentes níveis de acesso.
- **RF7** O sistema deve gerar relatórios de desempenho e histórico de falhas.
- **RF8** O sistema deve enviar notificações por e-mail ou aplicativo móvel.
- **RF9** O sistema deve permitir agendamento de manutenções preventivas.
- **RF10** O sistema deve exibir dashboard com indicadores (MTBF, MTTR, disponibilidade).

2.2. REQUISITOS NÃO FUNCIONAIS

- **RNF1** Desempenho: O sistema deve processar e exibir dados em até 3 segundos após a coleta.
- **RNF2** Disponibilidade: O sistema deve estar disponível 99% do tempo.
- **RNF3** Segurança: O sistema deve possuir autenticação por login e senha criptografada.
- **RNF4** Escalabilidade: O sistema deve suportar no mínimo 500 equipamentos cadastrados.
- **RNF5** Usabilidade: A interface deve ser intuitiva e permitir treinamento em no máximo 4 horas.
- **RNF6** Confiabilidade: Os dados devem possuir backup automático diário.
- **RNF7** Compatibilidade: O sistema deve funcionar em navegadores atualizados (Chrome, Edge e Firefox).
- **RNF8** Manutenibilidade: O sistema deve permitir atualização de software sem interromper o funcionamento por mais de 10 minutos.

3. PRIORIZAÇÃO

- ***MUST HAVE (Deve ter):*** Crítico. Sem isso, o projeto não é entregue ou não funciona. Essencial.
- ***SHOULD HAVE (Deveria ter):*** Importante, mas não vital. Pode ser entregue logo após o essencial. Alta Prioridade.
- ***COULD HAVE (Poderia ter):*** Desejável. É um diferencial que só será feito se sobrar tempo/recurso. Opcional.
- ***WON'T HAVE (Não terá):*** Fora de escopo para este momento. Ajuda a focar no que importa agora. Futuro.

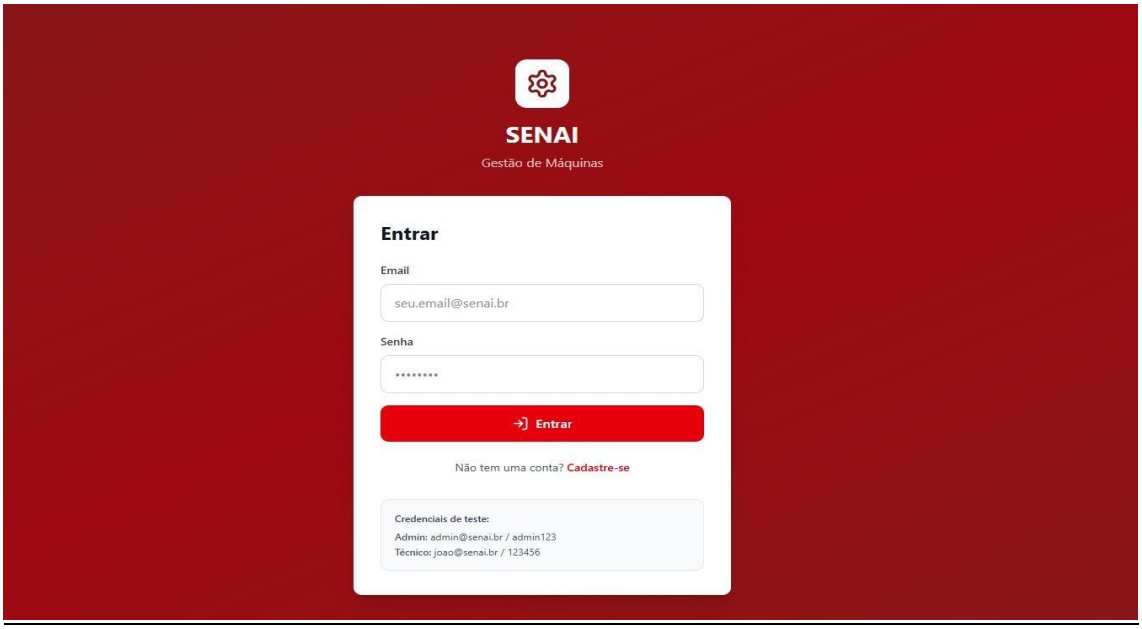
Requisito	**Priorização**	**Explicação**
RF1 – Coletar dados de sensores em tempo real	**MUST HAVE**	Essencial para o funcionamento do sistema de monitoramento preditivo.
RF2 – Armazenar dados em banco histórico	**MUST HAVE**	Necessário para análises, rastreabilidade e geração de relatórios.
RF3 – Análise automática com algoritmos preditivos	**MUST HAVE**	Principal diferencial do sistema de manutenção preditiva.
RF4 – Gerar alertas automáticos de anomalias	**MUST HAVE**	Fundamental para evitar falhas críticas e reduzir tempo de parada.
RF5 – Cadastro de máquinas e equipamentos	**MUST HAVE**	Base estrutural para organização e monitoramento dos ativos.
RF6 – Cadastro de usuários com níveis de acesso	**MUST HAVE**	Essencial para segurança e controle de permissões.
RF7 – Gerar relatórios de desempenho e falhas	**SHOULD HAVE**	Importante para análise estratégica e tomada de decisão.
RF8 – Enviar notificações por e-mail/app	**SHOULD HAVE**	Agrega agilidade na comunicação de eventos críticos.
RF9 – Agendamento de manutenções preventivas	**SHOULD HAVE**	Complementa a estratégia de manutenção preditiva.

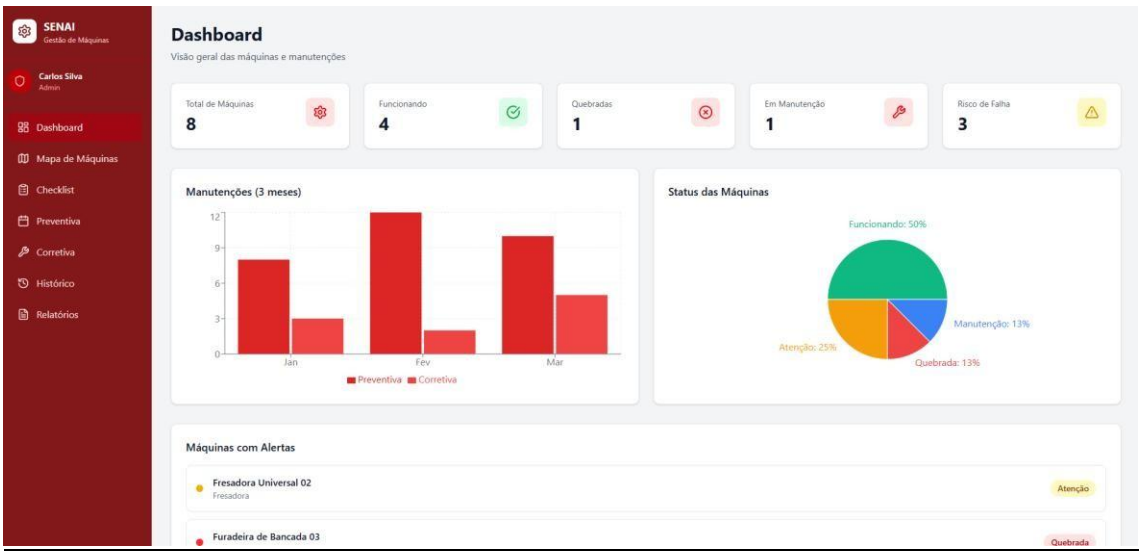
RF10 – Dashboard com indicadores (MTBF, MTTR, disponibilidade)	**SHOULD HAVE**	Importante para visualização gerencial e acompanhamento de desempenho.
RNF1 – Processamento em até 3 segundos	**MUST HAVE**	Requisito crítico de desempenho para garantir resposta em tempo real.
RNF2 – Disponibilidade de 99%	**MUST HAVE**	Garante confiabilidade mínima para operação industrial.
RNF3 – Autenticação com senha criptografada	**MUST HAVE**	Requisito essencial de segurança da informação.
RNF4 – Suportar 500 equipamentos	**SHOULD HAVE**	Importante para escalabilidade inicial do sistema.
RNF5 – Interface intuitiva (treinamento até 4h)	**SHOULD HAVE**	Melhora adoção e reduz custos de capacitação.
RNF6 – Backup automático diário	**MUST HAVE**	Essencial para garantir integridade e recuperação de dados.
RNF7 – Compatível com Chrome, Edge e Firefox	**COULD HAVE**	Importante para acessibilidade, mas não crítico na fase inicial.
RNF8 – Atualização com interrupção máxima de 10 min	**COULD HAVE**	Desejável para continuidade operacional, mas pode ser evoluído posteriormente.

4. FIGMA

Foi a peça-chave para validar a experiência do usuário (UX) e a interface (UI) antes do código. O protótipo de alta fidelidade no Figma garantiu o alinhamento visual com a marca SENAI, eliminou erros de usabilidade e acelerou o desenvolvimento ao servir como guia exato para a implementação técnica.

FIGMA





5. METODOLOGIAS ÁGEIS

São formas de gerenciar projetos focadas em entregas rápidas, adaptabilidade a mudanças e trabalho em equipe constante.

Metodologia Kanban

O Kanban é um método de gestão visual focado na eficiência do fluxo de trabalho.

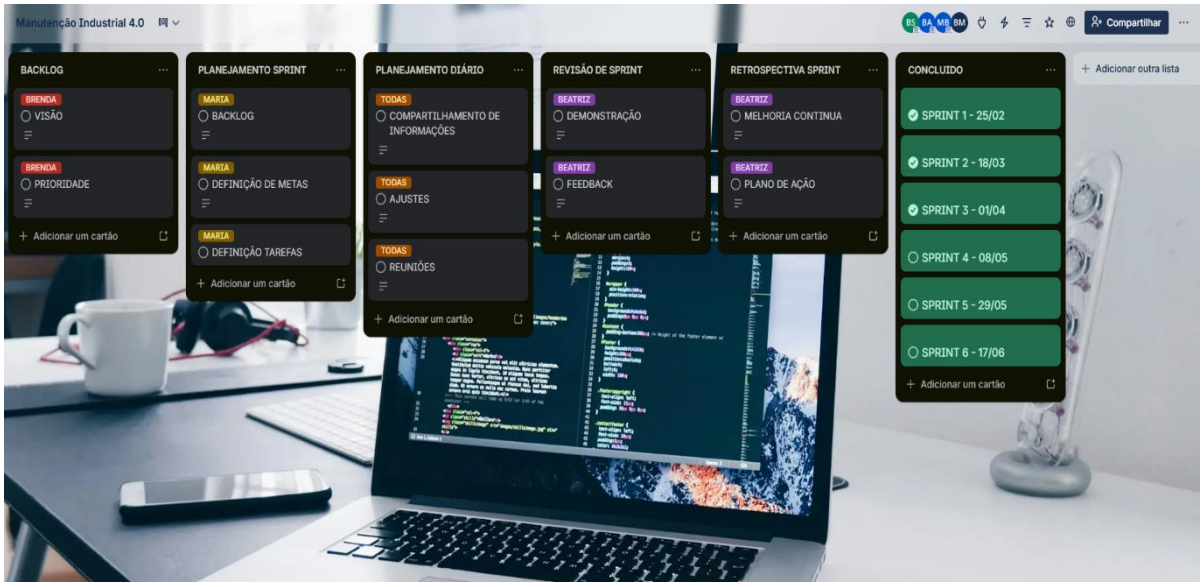
Utilizando cartões e colunas (como "A Fazer", "Em Execução" e "Feito"), ele permite que a equipe visualize o status de cada tarefa em tempo real.

O grande diferencial é a aplicação de limites de WIP (Work In Progress), que impedem a sobrecarga da equipe e garantem que o foco esteja na conclusão, não apenas no início de novas demandas.

Vantagens: Alta flexibilidade para mudanças de prioridade e transparência total do processo, facilitando a identificação imediata de gargalos.

Desvantagens: A ausência de prazos rígidos (sprints) pode dificultar a estimativa de grandes projetos a longo prazo. Além disso, exige alta disciplina da equipe para que o quadro reflita a realidade.

TRELLO



6. README e Versionamento (GIT)

No desenvolvimento do sistema para o Senai, a transição do físico para o digital exige clareza e segurança. Para garantir que o projeto seja sustentável e profissional, utilizamos duas ferramentas fundamentais: o arquivo README e o Versionamento (Git).

README

Manutenção Industrial 4.0 🏭

> Status do Projeto: 🚧 Em desenvolvimento

Equipe
BEATRIZ DOS SANTOS
BRENDA RIBEIRO ALVES
MARIA EDUARDA BEZERRA

🎯 Objetivo

Este projeto visa a digitalização completa do fluxo de manutenção fabril, substituindo registros físicos por uma inteligência de dados centralizada.

🔗 Links do Projeto

- 🎨 Protótipo no Figma
- 📄 Documentação Word
- 🗂️ Gestão no Trello

🌐 Funcionalidades e Tecnologias

- Definição do stack (Laravel/MySQL) e das funcionalidades core.
- Garante que o sistema seja robusto, padronizado e fácil de manter.

✔️ Levantamento Requisitos (Funcionais e Não Funcionais)

- Mapeamento do que o sistema faz e seus parâmetros de performance/segurança.
- Alinha a entrega técnica com a necessidade real do cliente.

📌 Priorização (MoSCoW)

- Filtro que separa o que é vital (Must Have) do que é opcional.
- Garante foco total na entrega do **MVP** (valor imediato).

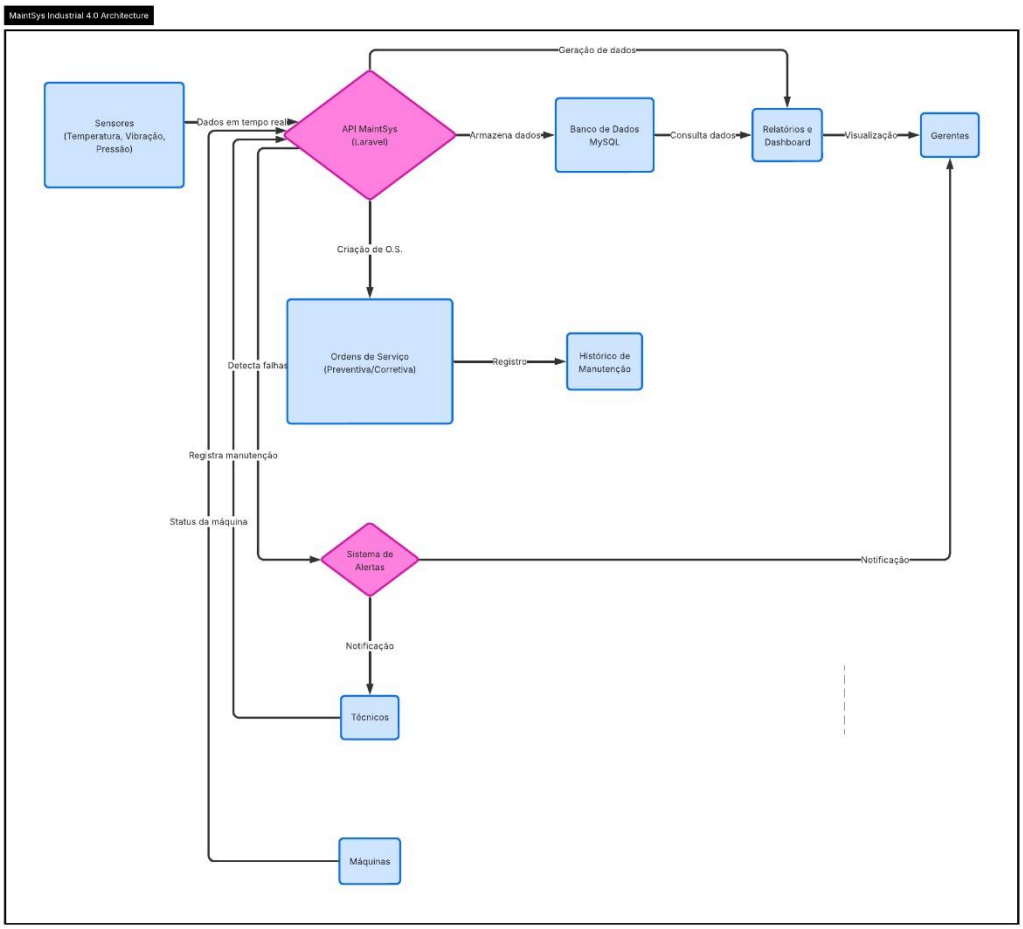
📋 Metodologia Kanban

- Gestão visual do fluxo de trabalho e das tarefas.
- Evita sobrecarga da equipe e dá transparência ao progresso do projeto.

🎨 Prototipagem

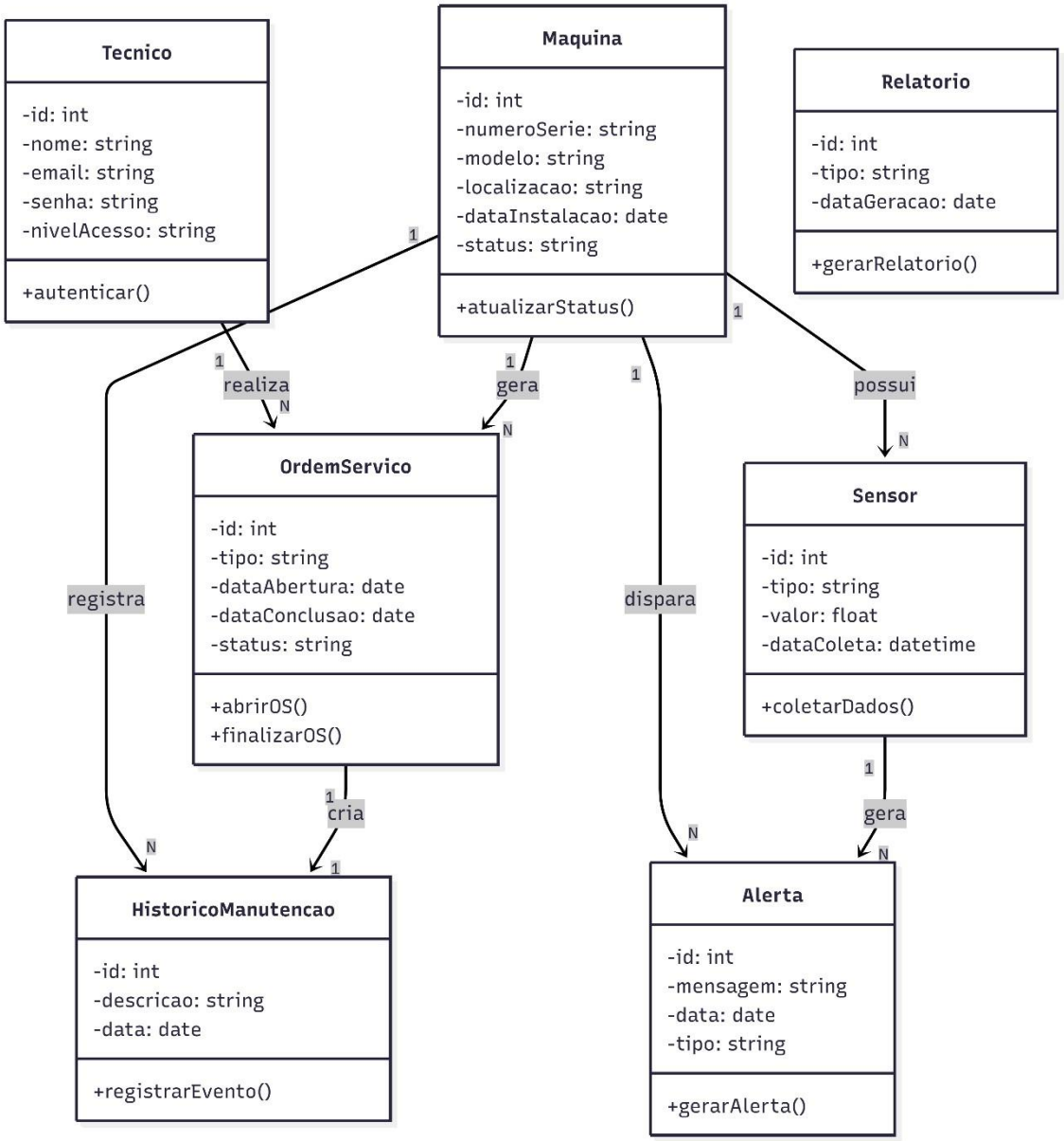
7. DIAGRAMA DE CLASSE

Este diagrama representa o ecossistema da solução, ilustrando o fluxo de dados desde a captura física nos sensores até a tomada de decisão gerencial.

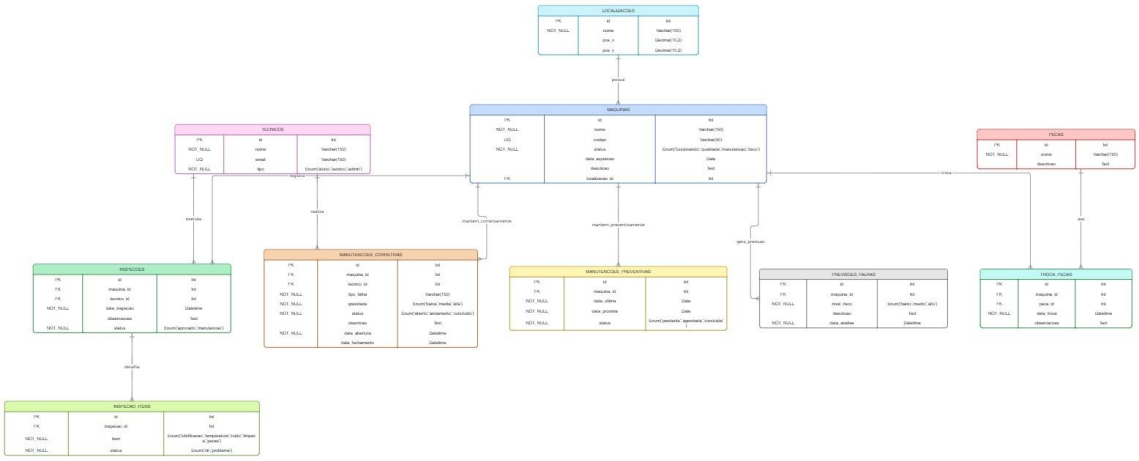


8. DIAGRAMA DE SEQUÊNCIA

O diagrama detalha a estrutura interna do sistema, definindo as propriedades e responsabilidades de cada objeto dentro da aplicação Laravel.



9. DIAGRAMA BANCO DE DADOS

Serve para visualizar, planejar e documentar a estrutura de dados de um sistema, mapeando tabelas, colunas e seus relacionamentos.

MIRO